



Sim. O melhor é separar esses projetos por **camadas**, porque alguns são código do sistema operacional, outros são desktop e outros existem principalmente para **montar a**

distribuição/ISO.

Um detalhe importante: a lista que montamos antes é uma **seleção dos componentes centrais**, não uma enumeração de todos os milhares de source packages do Ubuntu 26.04.

1. Kernel — linux

É o **kernel Linux utilizado pelo Ubuntu**, com o trabalho de integração/empacotamento do Ubuntu.

É a camada mais baixa do sistema operacional:

```
Aplicativos
  ↓
glibc / bibliotecas
  ↓
systemd / serviços
  ↓
Linux Kernel
  ↓
CPU / RAM / GPU / discos / USB / rede
```

Contém coisas como:

- scheduler;
- gerenciamento de memória;
- processos;
- filesystems;
- networking;
- drivers;
- namespaces;
- cgroups;
- segurança;
- suporte às arquiteturas;
- interfaces de system calls.

Para Resolute, o Ubuntu Kernel Team mantém o repositório correspondente.

Importância para criar seu Ubuntu: ★★★★★

2. Metapacotes do kernel — linux-meta

Isso não é outro kernel.

São metapacotes que fazem o APT apontar para a versão apropriada do kernel.

Por exemplo, conceitualmente:

```
linux-generic
  ↓ depende de
linux-image-X.Y.Z-generic
linux-headers-X.Y.Z-generic
```

Quando uma atualização de kernel chega, `linux-meta` é atualizado para fazer o sistema puxar a nova versão.

Importância: ★★☆☆☆

Base do sistema

3. base-files

Contém arquivos básicos que dão identidade e estrutura à distribuição.

Está relacionado a coisas como:

```
/etc/os-release
/etc/issue
/etc/debian_version
```

e determinados diretórios/arquivos fundamentais.

É aqui que aparecem várias características que fazem o sistema se identificar como Ubuntu.

Não contém o Ubuntu inteiro.

Importância: ★★★★★

4. APT — apt

É o gerenciador de pacotes de alto nível.

Fornece ferramentas como:

```
Bash
```

```
apt
apt-get
apt-cache
```

É responsável por resolver coisas como:

```
Quero instalar Firefox
↓
quais dependências?
↓
de qual repository?
↓
qual versão?
↓
download
↓
entrega ao dpkg
```

Também trabalha com:

```
/etc/apt/sources.list
/etc/apt/sources.list.d/
```

Importância: ★★★★★

5. DPKG — dpkg

É a camada inferior do sistema de pacotes Debian/Ubuntu.

Se APT é o gerente, dpkg é quem efetivamente manipula os `.deb`.

Por exemplo:

```
Bash
```

```
sudo dpkg -i pacote.deb
```

Ele mantém informações sobre pacotes instalados e executa scripts de instalação, atualização e remoção.

A relação é aproximadamente:

```
apt
  ↓
dpkg
  ↓
arquivos instalados no sistema
```

Importância: ★★★★★

6. GNU C Library — glibc

Uma das peças mais fundamentais de todo o userspace Linux.

Fornece a implementação da biblioteca C e interfaces usadas por uma quantidade enorme de programas:

```
programa
  ↓
glibc
  ↓
system calls
  ↓
Linux
```

Inclui funções como:

```
C

malloc()
free()
printf()
fopen()
socket()
```

e muitas outras interfaces POSIX/GNU.

Sem uma libc, praticamente todo o userspace tradicional precisaria ser construído de maneira completamente diferente.

Importância: ★★★★★

7. systemd

É o sistema de inicialização e gerenciamento de serviços.

Depois que o kernel sobe:

```
Firmware
↓
GRUB
↓
Kernel
↓
initramfs
↓
systemd
↓
serviços
↓
login/desktop
```

Ele controla unidades como:

```
Bash

systemctl start ssh
systemctl enable docker
systemctl status NetworkManager
```

O projeto também inclui vários componentes além do PID 1.

Importância: ★★★★★

8. Bash — bash

É o GNU Bourne Again Shell.

Fornece:

```
Bash

/bin/bash
```

É usado para terminal e principalmente uma quantidade enorme de scripts administrativos:

```
Bash
```

```
#!/usr/bin/env bash
```

```
for file in *; do
    echo "$file"
done
```

Não é o kernel nem o sistema inteiro, mas é parte importantíssima da experiência Unix/Linux.

Importância: ★★★★★☆

9. GNU Coreutils — coreutils

Aqui estão muitos dos comandos básicos que você usa constantemente.

Exemplos:

```
ls
cp
mv
rm
mkdir
cat
chmod
chown
touch
head
tail
sort
wc
date
```

Quando você digita:

```
Bash
```

```
ls
```

não é o Bash implementando `ls`. Normalmente é o executável GNU Coreutils.

Importância: ★★★★★

Boot

10. GRUB — grub2

É o bootloader.

Fluxo:

```
UEFI/BIOS
  ↓
GRUB
  ↓
Linux Kernel
  ↓
initramfs
  ↓
systemd
```

O GRUB localiza e carrega kernel/initramfs e oferece o menu de boot.

Importância: ★★★★★

11. initramfs-tools

Ferramentas utilizadas para gerar o **initramfs**.

O initramfs é um pequeno ambiente carregado na memória durante o boot.

Ele permite preparar coisas necessárias antes de montar o verdadeiro / .

Por exemplo:

```
Kernel
  ↓
initramfs
  ↓
carrega módulos
  ↓
localiza disco
  ↓
desbloqueia LUKS, se necessário
  ↓
monta root filesystem
  ↓
systemd
```

Importância: ★★★★★

Hardware

12. linux-firmware

Coleção enorme de blobs/firmwares necessários para diversos dispositivos.

Pode incluir firmware para:

- Wi-Fi;
- Bluetooth;
- GPUs;
- placas de rede;
- áudio;
- outros dispositivos.

Importante distinguir:

```
driver → geralmente kernel  
firmware → linux-firmware
```

Importância para desktop: ★★★★★

Gráficos

13. Mesa — mesa

É uma das maiores peças da stack gráfica Linux.

Implementa partes importantes de:

```
OpenGL  
Vulkan  
EGL  
GBM
```

e drivers userspace para várias GPUs.

Para AMD, por exemplo, você pode ter aproximadamente:

```
Aplicativo Vulkan
    ↓
RADV/Mesa
    ↓
amdgpu kernel driver
    ↓
GPU Radeon
```

Para OpenGL:

```
Aplicativo
    ↓
OpenGL/Mesa
    ↓
DRM
    ↓
Kernel
    ↓
GPU
```

Importância para desktop/games: ★★★★★

14. X.Org — xorg

Esse nome de source package não significa necessariamente "todo o código do X11 em um repositório". Boa parte da stack X.Org é dividida em vários pacotes.

O X11 tradicional envolve:

```
X Server
drivers
X libraries
protocols
utilities
```

O metapacote `xorg` ajuda a reunir componentes da stack.

Importância: ★★★☆☆

15. Wayland — wayland

Wayland fornece principalmente o **protocolo e bibliotecas fundamentais** da stack Wayland.

Não é sozinho o servidor gráfico equivalente ao Xorg.

No GNOME:

```
Aplicativo
↓
Wayland
↓
Mutter
↓
Mesa
↓
DRM/KMS
↓
Kernel
```

Importância para Ubuntu Desktop moderno: ★★★★★

GNOME Desktop

16. `gnome-shell`

É grande parte da interface gráfica que você vê no GNOME:

- painel superior;
- overview;
- workspaces;
- extensões;
- launcher;
- notificações;
- integração da sessão.

É uma das peças que mais influencia a aparência do Ubuntu Desktop.

Importância para Ubuntu Desktop: ★★★★★

17. **Mutter** — `mutter`

É o compositor/window manager do GNOME.

Responsável por coisas como:

- janelas
- composição
- monitores
- workspaces
- wayland
- input
- renderização/composição

GNOME Shell e Mutter trabalham intimamente juntos.

Simplificando:

- GNOME Shell → interface
- Mutter → compositor/window manager

Importância: ★★★★★

18. gnome-control-center

É o aplicativo **Configurações** do GNOME.

Fornece painéis para coisas como:

- Displays
- Network
- Bluetooth
- Sound
- Users
- Keyboard
- Mouse
- Power
- Accessibility

Não são "arquivos GNOME" genericamente; uma descrição melhor seria:

Painel de configurações do GNOME.

Importância: ★★★★★☆

19. Nautilus — nautilus

É o gerenciador de arquivos do GNOME.

É o aplicativo normalmente chamado simplesmente de **Files**.

Controla:

- navegação por diretórios;
- copiar/mover;
- dispositivos;
- compartilhamentos;
- thumbnails;
- integração com o desktop GNOME.

Importância: ★★★★★☆

20. GDM — gdm3

GNOME Display Manager.

É a tela gráfica de login.

```
boot
  ↓
systemd
  ↓
gdm
  ↓
login
  ↓
GNOME session
  ↓
GNOME Shell
```

Também inicia/gerencia sessões gráficas.

Importância: ★★★★★★

Rede

21. NetworkManager — network-manager

Gerencia conectividade de rede.

Controla:

Ethernet
Wi-Fi
VPN
DNS
interfaces
rotas

É utilizado pelo GNOME e por:

Bash
nmcli

Importância para desktop: ★★★★★

Áudio

22. PipeWire — pipewire

Stack moderna para áudio e vídeo multimídia.

Hoje ocupa funções historicamente realizadas por diferentes subsistemas.

Fluxo aproximado:

Aplicativo
↓
PipeWire
↓
ALSA
↓
Kernel
↓
hardware

Também é muito importante para captura/compartilhamento de tela em Wayland.

Importância: ★★★★★

23. PulseAudio — pulseaudio

Historicamente foi o servidor de áudio padrão de muitas distribuições.

Em sistemas modernos, parte da compatibilidade PulseAudio pode ser fornecida através do PipeWire.

Então aplicativos que esperam:

PulseAudio API

podem continuar funcionando enquanto PipeWire está por baixo.

Importância atual: ★★☆☆☆

Bluetooth

24. BlueZ — bluez

É a implementação principal da stack Bluetooth Linux em userspace.

Trabalha com o suporte Bluetooth do kernel.

Fornece serviços e ferramentas para:

headsets
teclados
mouse
controles
Bluetooth LE

Importância: ★★★★★

Dispositivos

25. udev — dentro de systemd

Hoje o udev faz parte do projeto systemd.

É responsável por detectar/gerenciar eventos de dispositivos.

Exemplo:

```

você conecta pendrive
      ↓
kernel detecta USB
      ↓
udev recebe evento
      ↓
cria/configura device
      ↓
desktop pode reagir
```

Está relacionado aos dispositivos encontrados em:

```
/dev/
```

Importância: ★★★★★

Administração

26. sudo

Permite executar comandos com privilégios de outro usuário, normalmente root:

```
Bash
```

```
sudo apt update
```

Controla políticas através de:

```
/etc/sudoers
/etc/sudoers.d/
```

Importância: ★★★★★☆

27. OpenSSH — openssh

Implementação de SSH utilizada no Ubuntu.

Inclui cliente:

```
Bash
```

```
ssh usuario@servidor
```

e servidor:

```
sshd
```

Além de ferramentas relacionadas como `scp` e `sftp`.

Importância: ★★★★★☆

Linguagens/toolchain

28. `python3-defaults`

Aqui há outra distinção importante.

`python3-defaults` **não contém simplesmente todo o código do interpretador Python.**

Ele é parte da infraestrutura Debian/Ubuntu que define qual versão Python 3 é a padrão e fornece metapacotes relacionados.

Por exemplo:

```
python3
  ↓
versão Python padrão da distribuição
```

O código do CPython propriamente dito está nos pacotes correspondentes às versões do Python.

Importância: ★★★★★☆☆

29. **GCC** — `gcc-*`

GNU Compiler Collection.

Compila:

```
C
C++
Fortran
...
```

É parte fundamental da toolchain usada para construir uma enorme quantidade do próprio Ubuntu.

A toolchain completa também envolve:

```
gcc
binutils
glibc
make
headers
linker
```

E a versão concreta depende da release e da configuração da distribuição; portanto não trataria `gcc-15` como universalmente "o GCC do Ubuntu 26" sem verificar o archive atual.

Importância para recompilar o sistema: ★★★★★

Instalação

30. Subiquity — subiquity

Instalador desenvolvido pela Canonical, especialmente importante para Ubuntu Server e infraestrutura moderna de instalação.

Cuida de etapas como:

```
discos
particionamento
usuários
rede
storage
instalação
```

Importância para Server: ★★★★★

31. ubuntu-desktop-provision

Componente do processo moderno de provisionamento/instalação do Ubuntu Desktop.

Trabalha na experiência gráfica de configuração/instalação do sistema.

Se você pretende criar uma derivação moderna do Ubuntu Desktop, é um projeto interessante.

Importância para Desktop ISO: ★★★★★

Construção da Live ISO

Esses próximos são particularmente importantes **para o que você quer fazer**.

32. livecd-rootfs

Constrói/customiza o filesystem utilizado pelas imagens live.

Conceitualmente:

```
Ubuntu archive
  ↓
livecd-rootfs
  ↓
instala pacotes
  ↓
configura filesystem
  ↓
root filesystem da imagem
```

É uma das peças centrais para construir imagens Ubuntu.

Para seu projeto: ★★★★★

33. Casper — casper

Faz o Ubuntu funcionar como um **sistema Live**.

Ou seja:

```
pendrive/DVD
  ↓
boot
  ↓
Ubuntu funcionando sem instalação
```

Ele cuida de vários aspectos da inicialização do ambiente live e localização/montagem do filesystem live.

É uma das razões pelas quais você consegue escolher algo equivalente a:

Try Ubuntu

antes de instalar.

Importância para Live ISO: ★★★★★

34. ubuntu-meta

Extremamente importante para entender como Ubuntu é composto.

São **metapacotes** que expressam conjuntos de dependências.

Por exemplo, conceitualmente:

```
ubuntu-desktop
  ↓
GNOME
NetworkManager
PipeWire
Firefox
fonts
drivers
utilities
...
```

Ele não contém todo esse código. Ele **declara o que deve ser instalado**.

Pense nele como uma lista/receita de componentes da edição Ubuntu.

Importância para sua distro: ★★★★★

35. ubuntu-cdimage

Infraestrutura/orquestração utilizada para produzir imagens oficiais Ubuntu.

Coordena partes do processo de construção/publicação das imagens.

Pense:

```

ubuntu-cdimage
  ↓
coordena build
  ↓
filesystem
  +
pacotes
  +
metadados
  +
boot
  ↓
imagem

```

Não é o código do Ubuntu propriamente dito; é **infraestrutura para fabricar imagens Ubuntu**.

Importância para reproduzir processo oficial: ★★★★★

36. Germinate — germinate

Ferramenta utilizada historicamente pelo Ubuntu para processar **seeds** e dependências.

Você pode pensar em seeds como listas que dizem:

```

Queremos:
GNOME
Firefox
NetworkManager
PipeWire
...

```

Germinate considera dependências e transforma essa seleção em conjuntos de pacotes.

Simplificando:

```

SEEDS
  ↓
germinate
  ↓
dependências
  ↓
lista de pacotes

```

É muito relevante para entender **como uma flavor/edição é composta**.

Importância: ★★★★★☆

37. debian-cd

Ferramentas originadas no Debian para criação de imagens de instalação.

No pipeline de imagens, participa da preparação da estrutura que termina na mídia bootável.

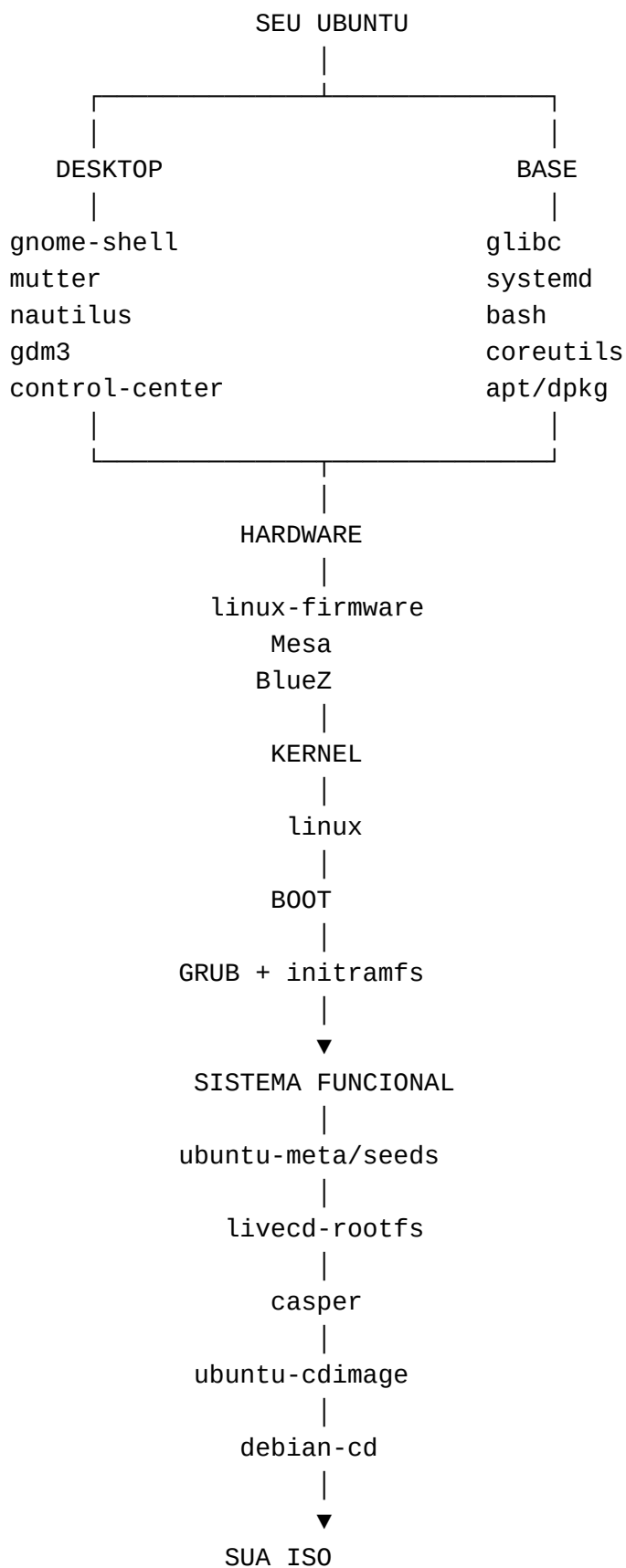
Pode lidar com elementos relacionados a:

- estrutura ISO
- pool de pacotes
- boot
- metadados
- checksums

Importância para geração da ISO: ★★★★★

O mapa que importa para o que você quer fazer

Sua ideia fica muito mais clara olhando assim:



E aqui está a distinção mais importante para seu projeto:

Código do sistema:

```
linux
glibc
systemd
bash
coreutils
apt
dpkg
grub2
mesa
gnome-shell
mutter
...
```

Receita/composição do Ubuntu:

```
ubuntu-meta
seeds
germinate
```

Ferramentas para fabricar a imagem:

```
livecd-rootfs
casper
ubuntu-cdimage
debian-cd
```

Portanto, para fazer um **Ubuntu 26 realmente seu**, eu não tentaria modificar os 37 projetos de uma vez. Eu começaria pela **receita (ubuntu-meta /seeds) + construção (livecd-rootfs)**, e clonaria `linux` , `systemd` , `mesa` , `GNOME` etc. somente quando você quiser substituir/recompilar aquele componente específico.



1. Kernel — `linux`

Agora o `/sys` do `freeBSD` é equivalente? Em termos de funcionalidades?

Sim — **conceitualmente, o `sys/` do source tree do FreeBSD é aproximadamente equivalente ao repositório `linux` do Ubuntu/Linux**, mas a organização interna é bastante diferente.

No FreeBSD:

FreeBSD	src/	
└─	sys/	← kernel
├─	amd64/	
├─	arm/	
├─	arm64/	
├─	dev/	← drivers
├─	fs/	← filesystems
├─	kern/	← núcleo
├─	net/	← networking
├─	netinet/	
├─	security/	
├─	sys/	← headers/interfaces do kernel
└─	vm/	← memória virtual

No Linux:

```
linux/
├─ arch/           ← arquiteturas
├─ block/          ← block layer
├─ crypto/
├─ drivers/        ← drivers
├─ fs/             ← filesystems
├─ include/        ← headers
├─ init/
├─ ipc/
├─ kernel/         ← núcleo
├─ mm/             ← memória
├─ net/            ← networking
├─ security/
├─ sound/
├─ tools/
└─ virt/
```

Equivalências aproximadas

FreeBSD	sys/	Linux	linux/	Função
	sys/kern/		kernel/	núcleo, processos, scheduler etc.
	sys/vm/		mm/	memória virtual
	sys/dev/		drivers/	drivers
	sys/fs/		fs/	filesystems

FreeBSD	sys/	Linux	linux/	Função
	sys/net/		net/	rede
	sys/netinet/		net/ipv4 , partes de net/	TCP/IP
	sys/security/		security/	mecanismos de segurança
	sys/amd64/		arch/x86/	arquitetura x86-64
	sys/arm64/		arch/arm64/	ARM64
	sys/sys/		include/linux/ + outros	interfaces/headers
partes de	sys/geom/		block/ + drivers/ block/ etc.	armazenamento/blocos

Então, em uma visão simplificada:

FreeBSD		Linux
src/sys/	≈	linux.git
├ kern/	≈	├ kernel/
├ vm/	≈	├ mm/
├ dev/	≈	├ drivers/
├ fs/	≈	├ fs/
├ net/	≈	├ net/
├ security/	≈	├ security/
└ arm64/	≈	└ arch/arm64/

Mas há uma diferença **fundamental**.

No FreeBSD:

src/	
├ sys/	← kernel
├ bin/	← userspace
├ lib/	
├ sbin/	
├ usr.bin/	
└ usr.sbin/	

O projeto FreeBSD desenvolve **kernel + grande parte do sistema-base juntos**.

No Ubuntu/Linux:

linux.git	← somente kernel
glibc	← separado
systemd	← separado
bash	← separado
coreutils	← separado
util-linux	← separado
iproute2	← separado
openssh	← separado
...	

Por isso o `sys/` do FreeBSD parece estar "dentro de um sistema operacional", enquanto o Git do Linux parece isolado. **Essa percepção está correta.**

Em termos de **responsabilidade de kernel**, entretanto:

FreeBSD `src/sys/` $\approx$ **Linux kernel source tree.**

Ambos implementam as funções centrais de um kernel: processos, memória virtual, drivers, filesystems, rede, IPC, segurança, suporte de arquitetura, interrupções, timers, SMP etc.

Eles não são equivalentes linha por linha nem possuem exatamente as mesmas funcionalidades/arquiteturas internas — FreeBSD e Linux tomaram muitas decisões de projeto diferentes — mas ocupam **a mesma camada do sistema operacional.**

Isso também explica exatamente o que você estava procurando antes: se quiser comparar o **FreeBSD inteiro** com Ubuntu, `src/sys/` ↔ `linux` é só a primeira peça. `src/bin`, `src/sbin`, `src/lib`, `src/usr.bin` etc. precisam ser comparados com `glibc`, `coreutils`, `util-linux`, `systemd` e vários outros projetos do ecossistema GNU/Linux.



bom acho melhor você comparar o kernel do linux com o source todo do freeBSD, porquê pelo jeito dentro do kernel do linux tem o kernel do kernel, olha lá o nome do diretório.

Exatamente — essa comparação é mais interessante, mas com uma correção importante:

o diretório `linux/kernel/` **não é “o kernel inteiro dentro do kernel”**. O repositório Linux inteiro é o kernel; `kernel/` é apenas uma das subdivisões internas.

Pense assim:

```
linux/                                ← TODO o kernel Linux
├─ arch/
├─ block/
├─ drivers/
├─ fs/
├─ kernel/                            ← núcleo genérico de várias funções
├─ mm/
├─ net/
├─ security/
└─ ...
```

Enquanto o FreeBSD é:

```
freebsd-src/                          ← kernel + sistema-base
├─ sys/                               ← kernel FreeBSD
├─ bin/
├─ lib/
├─ sbin/
├─ usr.bin/
├─ usr.sbin/
└─ ...
```

Linux kernel vs FreeBSD source inteiro

Linux	<code>linux/</code>	FreeBSD	<code>src/</code>	Comparação
	<code>kernel/</code>		<code>sys/kern/</code>	núcleo genérico
	<code>mm/</code>		<code>sys/vm/</code>	memória virtual
	<code>drivers/</code>		<code>sys/dev/</code> + outros	drivers
	<code>fs/</code>		<code>sys/fs/</code> + partes de <code>sys/</code>	filesystems
	<code>net/</code>		<code>sys/net*</code>	networking
	<code>arch/</code>		<code>sys/amd64</code> , <code>sys/arm64</code> etc.	arquiteturas
	<code>security/</code>		<code>sys/security/</code>	segurança
	<code>block/</code>		<code>sys/geom/</code> + armazenamento	block/storage

Linux	linux/	FreeBSD	src/	Comparação
	sound/		sys/dev/sound/	áudio
	virt/		partes de sys/	virtualização
	include/		sys/sys/ , sys/*/include etc.	headers
	tools/		tools/ + algumas ferramentas	desenvolvimento/testes
	scripts/		sys/conf/ , tools/ , Makefiles	build/configuração
—			bin/	userspace FreeBSD
—			sbin/	userspace/administração
—			usr.bin/	programas userspace
—			usr.sbin/	programas administrativos
—			lib/	bibliotecas do sistema

E aí aparece a diferença que você percebeu.

O linux/ é enorme porque o Linux coloca muita coisa no kernel

Por exemplo:

```
linux/
├─ drivers/
│   ├── gpu/
│   ├── net/
│   ├── usb/
│   ├── nvme/
│   ├── bluetooth/
│   └─ ...
├─ fs/
│   ├── ext4/
│   ├── btrfs/
│   ├── nfs/
│   └─ ...
├─ net/
├─ sound/
├─ crypto/
├─ security/
└─ virt/
```

Isso pode dar a impressão de:

“Caramba, esse repositório já parece um sistema operacional inteiro.”

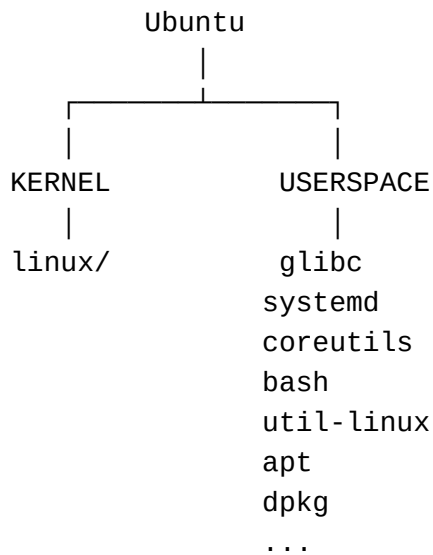
E em relação ao **lado kernel/hardware**, realmente é gigantesco.

Mas falta uma coisa essencial: **userspace**.

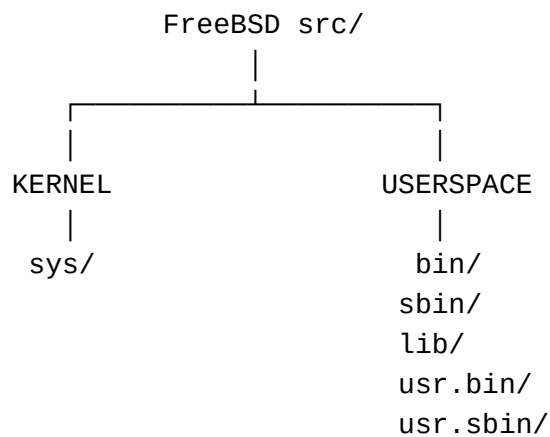
O kernel Linux sozinho não fornece uma experiência como:

```
/bin/bash
/bin/ls
/usr/bin/cp
/usr/bin/ssh
apt
systemctl
GNOME
login
sudo
```

Um sistema Linux funcional combina:



Já FreeBSD é desenvolvido mais como:



E sobre linux/kernel/

Esse nome realmente pode confundir.

Quando você clona:

Bash

```
git clone https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.g
```

você recebe:

```
linux/                                     ← KERNEL COMPLETO
|
├─ kernel/                               ← funcionalidades centrais/genéricas
|   ├─ sched/
|   ├─ locking/
|   ├─ irq/
|   ├─ time/
|   ├─ cgroup/
|   ├─ events/
|   └─ ...
|
├─ mm/                                   ← memória
├─ drivers/                             ← drivers
├─ fs/                                  ← filesystems
├─ net/                                 ← rede
├─ arch/                                ← CPU/arquitetura
└─ ...
```

Portanto:

```
linux/
  ↓
kernel inteiro

linux/kernel/
  ↓
uma parte do kernel
```

No FreeBSD é quase a mesma ideia:

```
freebsd-src/sys/
  ↓
kernel inteiro

freebsd-src/sys/kern/
  ↓
uma parte central do kernel
```

A comparação mais precisa é:

Linux

FreeBSD

linux/	≈	freebsd-src/sys/
├─ kernel/	≈	├─ kern/
├─ mm/	≈	├─ vm/
├─ drivers/	≈	├─ dev/
├─ fs/	≈	├─ fs/
├─ net/	≈	├─ net*/
└─ arch/	≈	└─ amd64/, arm64/...

Não:

```
linux/kernel/ ≈ freebsd-src/sys/
```

mas sim:

```
linux/ ≈ freebsd-src/sys/
```

E aí vem a conclusão importante para o que você está investigando: **o freebsd-src inteiro é maior conceitualmente que o kernel Linux**, porque o FreeBSD src inclui seu kernel e grande parte do userspace/base OS.

Se quisermos fazer uma comparação realmente boa para o seu objetivo, o próximo passo seria comparar **cada diretório do freebsd-src/ (bin , lib , sbin , sys , usr.bin , usr.sbin ...)** com os projetos equivalentes do **Ubuntu/Linux**. Aí finalmente conseguimos descobrir quais repositórios você precisaria juntar para criar um “ ubuntu-src ” equivalente ao freebsd-src .



O ChatGPT pode cometer erros. Por isso, lembre-se de conferir informações relevantes.